

Report On

Face Recognition Attendance Management System

Submitted in partial fulfillment of the requirements of the Mini project in Semester IV
of Second Year Computer Science Engineering (Data Science)

by

Parth Paliwal (Roll No. 47)

Harshal Nivate (Roll No. 46)

Moksh Patel (Roll No. 57)

Parth Nalang (Roll No. 42)

Mentor

Asst. Prof. Shital Cheke

University of Mumbai



Vidyavardhini's College of Engineering & Technology

Department of Computer Science Engineering (Data Science)



(A.Y. 2025-26)

Vidyavardhini's College of Engineering & Technology
Department of Computer Science Engineering (Data Science)

CERTIFICATE

This is to certify that the Mini Project entitled "Face Recognition Attendance Management System" is a bonafide work of Parth Paliwal (47), Harshal Nivate (46), Moksh Patel (57), Parth Nalang (42) submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of "Bachelor of Engineering" in Semester IV of Second Year "Computer Science Engineering (Data Science)".

Asst. Prof. Shital Cheke

Mentor

Prof. Satish Salunkhe (Ph.D.)

Head of Department

Dr. Rakesh Himte

Principal

Vidyavardhini's College of Engineering & Technology
Department of Computer Science Engineering (Data Science)

Mini Project Approval

This Mini Project entitled "Face Recognition Attendance Management System" by Parth Paliwal (47), Harshal Nivate (46), Moksh Patel (57), Parth Nalang (42) is approved for the degree of Bachelor of Engineering in Semester IV of Second Year Computer Science Engineering (Data Science).

Examiners

.....

(Internal Examiner Name & Sign)

.....

(External Examiner Name & Sign)

Date: _____

Abstract

This report presents the design, implementation, and evaluation of a Face Recognition Attendance Management System developed using Python, OpenCV, Tkinter, and MySQL. Traditional attendance systems — such as manual registers or ID card swipes — are susceptible to proxy attendance, human error, and significant administrative overhead. The proposed system addresses these shortcomings by leveraging computer vision and biometric recognition to automate the entire attendance process in a contactless and tamper-resistant manner.

The system captures and registers student facial images through a webcam and trains a Local Binary Pattern Histogram (LBPH) face recognition model. During attendance sessions, the webcam feed is processed in real time; recognized students are automatically logged in CSV files with name, roll number, department, time, and date. A three-layer anti-spoofing mechanism — combining Laplacian blur detection, screen glare analysis, and YCrCb skin-colour verification — prevents fraudulent attendance using photographs or phone screens.

The graphical interface provides modules for student registration, model training, attendance record management (import, update, export), and an SMTP-based email notification system for absent students. The system uses a login module with MySQL credential verification and a hardcoded fallback. Testing on four registered students demonstrated approximately 92% recognition accuracy under standard lighting, with attendance marking latency below 200 milliseconds.

The report concludes with an analysis of current limitations and proposes future enhancements including deep learning-based recognition (FaceNet/ArcFace), a web-based dashboard, direct MySQL attendance logging, and secure credential management.

Acknowledgement

We would like to thank all people whose support and cooperation has been an invaluable asset during this project. We convey our sincere gratitude to our Guide, Prof. [Name Surname], for guiding us throughout this project and giving it the present shape. It would have been impossible to complete the project without the support, valuable suggestions, criticism, encouragement, and guidance extended to us.

We convey our gratitude to Prof. Satish Salunkhe (Ph.D.), Head of Department, for his motivation and for providing various facilities, which helped us greatly in the whole process of this project. We are also grateful to all other teaching and non-teaching staff members of the Computer Science Engineering (Data Science) Department for directly or indirectly helping us with the completion of this project and the resources provided.

Parth Paliwal (Roll No. 47)

Harshal Nivate (Roll No. 46)

Moksh Patel (Roll No. 57)

Parth Nalang (Roll No. 42)

Date: _____

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
CV	Computer Vision
LBPH	Local Binary Pattern Histogram
GUI	Graphical User Interface
CSV	Comma Separated Values
MySQL	My Structured Query Language
SMTP	Simple Mail Transfer Protocol
MIME	Multipurpose Internet Mail Extensions
XML	eXtensible Markup Language
PIL	Python Imaging Library
ROI	Region of Interest
YCrCb	Luminance, Chroma Red, Chroma Blue (colour space)
RFID	Radio Frequency Identification
PCA	Principal Component Analysis
FPS	Frames Per Second
AY	Academic Year

List of Figures

Figure No.	Title	Page No.
Fig. 3.1	Login Screen – Screenshot	16
Fig. 3.2	Main Dashboard – Screenshot	16
Fig. 3.3	System Architecture Block Diagram	17
Fig. 3.4	Student Management System – Screenshot	18
Fig. 3.5	Train Data Set – Screenshot	18
Fig. 3.6	Face Recognition Module – Screenshot	19
Fig. 3.7	Attendance Management System – Screenshot	19
Fig. 3.8	Email Sender – Low Attendance Alert – Screenshot	20
Fig. 3.10	MySQL Student Table Schema	20
Fig. 3.10	MySQL Student Database Records	21
Fig. 3.11	Face Recognition Live Detection – Screenshot	21

List of Tables

Table No.	Title	Page No.
Table 2.1	Comparison of Existing Attendance Systems	13
Table 3.1	Technology Stack	15
Table 3.2	MySQL student Table Schema	16
Table 3.3	Hardware Requirements	23
Table 3.4	Software Requirements	23
Table 3.5	Unit Testing Results	24
Table 3.6	System Performance Metrics	25

Contents

Section	Page No.
Abstract	i
Acknowledgement	ii
List of Abbreviations	iii
List of Figures	iv
List of Tables	v
Chapter 1: Introduction	1
1.1 Introduction	1
1.2 Problem Statement & Objectives	2
1.3 Project Scope	3
Chapter 2: Literature Review	4
2.1 Survey of Existing Systems / SRS	4
2.2 Limitation of Existing System / Research Gap	7
2.3 Mini Project Contribution	8
Chapter 3: Proposed System	9
3.1 Introduction	9
3.2 Architecture / Block Diagram	10
3.3 Algorithm and Process Design	11
3.4 Details of Hardware & Software	16
3.5 Experiment and Results for Validation and Verification	17
3.6 Analysis	19
3.7 Conclusion and Future Work	20
References	22
Annexure	24

Chapter 1

Introduction

1.1 Introduction

Face recognition technology has emerged as one of the most effective and widely adopted biometric identification methods in modern computing. It operates by analysing and matching distinct facial features — such as the distance between eyes, nose shape, and jawline geometry — to identify or verify individuals in real time. Over the last decade, advancements in open-source computer vision libraries and affordable webcam hardware have made face recognition a cost-effective solution for a wide range of applications including security systems, access control, and attendance management.

Traditional attendance tracking mechanisms such as manual registers, ID card swipes, and PIN-based entry systems suffer from several inherent limitations. The most significant of these is proxy attendance, commonly known as "buddy punching," where one student marks attendance on behalf of another. Manual systems are also time-consuming for large batches, produce records that are difficult to archive and query, and are prone to transcription errors. These problems are significantly amplified in institutions where attendance must be recorded for hundreds of students across multiple sessions each day.

The Face Recognition Attendance Management System presented in this report addresses all of these shortcomings by leveraging computer vision and machine learning to provide automated, contactless, and tamper-proof attendance recording. The system is built entirely in Python, integrating OpenCV's Local Binary Pattern Histogram (LBPH) face recognition algorithm, a Tkinter-based graphical user interface, a MySQL relational database for student data storage, CSV files for portable attendance records, and an SMTP-based email notification module for absence alerts.

1.2 Problem Statement & Objectives

Problem Statement:

Existing attendance systems in educational institutions rely on manual processes that are time-consuming, error-prone, and susceptible to fraudulent entries. There is a

pressing need for an automated, biometric-based attendance system that accurately records attendance in real time, prevents proxy entries, generates reliable digital records, and provides an easy-to-use administrative interface.

Aim:

To design and implement a Python-based face recognition attendance management system that automates student attendance recording using webcam-based biometric identification, with secure login, anti-spoofing protection, administrative attendance management, and email notification capabilities.

Objectives:

- To implement automated real-time face detection and recognition using the OpenCV Haar cascade and LBPH algorithms.
- To develop a student registration module that captures facial images and stores student records in a MySQL database.
- To train and persist an LBPH face recognition model from captured facial image datasets.
- To implement date-scoped duplicate prevention logic ensuring each student is marked only once per attendance session.
- To incorporate a three-layer anti-spoofing mechanism to detect and reject fraudulent attendance attempts using photographs or digital screens.
- To provide an attendance management interface for importing, viewing, updating, and exporting attendance records.
- To implement an SMTP-based email notification system for alerting guardians or students regarding absenteeism.
- To secure the application with a role-based login module supporting both MySQL credential verification and hardcoded fallback credentials.

1.3 Project Scope

The scope of this mini project is limited to a desktop application deployed on Windows-based systems equipped with a USB or built-in webcam. The system is intended for use by an administrator — typically a faculty member or institution staff — who manages

student enrolment, model training, and attendance oversight. The project scope specifically includes:

- Registration and management of student biometric profiles (facial images) using a standard webcam.
- Offline face recognition model training using the LBPH algorithm — no internet connection is required for core recognition functionality.
- Real-time attendance marking via a live webcam feed with integrated anti-spoofing checks.
- CSV-based attendance record management including import, status update, and export functions.
- Email notifications for absent students using the Gmail SMTP service.

The scope does not include mobile application development, cloud-based deployment, integration with existing institutional ERP systems, recognition of masked individuals, or operation under extreme lighting conditions.

Chapter 2

Literature Review

2.1 Survey of Existing Systems / SRS

A substantial body of research exists in the domain of automated attendance systems using both biometric and non-biometric technologies. This section surveys the principal approaches and their characteristics:

2.1.1 Manual and Semi-Automated Systems

Manual attendance registers remain the most widely used method in educational institutions owing to their simplicity and zero infrastructure cost. However, they are inherently susceptible to proxy entries, require significant faculty time, and produce paper records that are difficult to archive, search, or report on. Semi-automated systems using barcode or RFID card readers improve recording speed but remain vulnerable to card sharing and require dedicated hardware for each access point.

2.1.2 Fingerprint-Based Biometric Systems

Fingerprint recognition systems offer very high identification accuracy (typically above 99%) and are widely deployed in corporate environments. However, they require dedicated scanner hardware (approximate cost: Rs. 2,000 – Rs. 10,000 per device), cannot reliably function with wet, worn, or injured fingertips, and raise significant hygiene concerns — particularly relevant in post-pandemic educational environments.

2.1.3 Face Recognition Using Eigenfaces (PCA)

Eigenface-based recognition, introduced by Turk and Pentland (1991), applies Principal Component Analysis (PCA) to represent faces as linear combinations of eigenvectors computed from a training set. While computationally efficient for small datasets, Eigenfaces perform poorly under lighting variation and are sensitive to changes in facial expression or head orientation, making them unsuitable for real-world deployment without strictly controlled imaging conditions.

2.1.4 Local Binary Pattern Histogram (LBPH) Recognition

The LBPH approach, introduced by Ahonen, Hadid, and Pietikainen (2006), extracts local texture descriptors that are inherently robust to monotonic illumination changes. Multiple studies have demonstrated LBPH achieving recognition accuracy between 88% and 95% for small-to-medium datasets under varied lighting. The OpenCV library provides a production-ready LBPH implementation via the `cv2.face` module, making it highly suitable for Python-based academic project implementations that require no GPU hardware.

2.1.5 Deep Learning-Based Face Recognition

Modern approaches such as FaceNet (Schroff et al., 2015) and ArcFace (Deng et al., 2019) apply deep convolutional neural networks to produce high-dimensional face embeddings, achieving recognition accuracy above 99.7% on standard benchmarks. However, these models require GPU hardware for real-time deployment, large labelled training datasets, and substantially greater computational resources — placing them beyond the scope of a lightweight, webcam-based desktop attendance system for academic institutions.

Table 2.1: Comparison of Existing Attendance Systems

Method	Accuracy	HW Cost	Proxy Prevention	Complexity
Manual Register	N/A	Nil	None	Low
RFID / Barcode	~99%	Medium	Partial	Low
Fingerprint	>99%	High	High	Medium
Eigenface (PCA)	~80%	Low	High	Medium
LBPH (Proposed)	~92%	Low	High	Medium
Deep CNN (FaceNet)	>99%	High (GPU)	High	High

2.2 Limitation of Existing System / Research Gap

Based on the survey presented above, the following limitations and research gaps are identified:

- Manual and RFID systems provide no biometric verification and are easily defeated by proxy attendance — the fundamental problem motivating this project.

- Fingerprint-based systems require expensive dedicated hardware per access point, raise hygiene concerns, and fail reliably with wet or damaged fingertips.
- Eigenface (PCA) systems perform poorly without controlled and uniform lighting conditions and are highly sensitive to changes in facial expression and head orientation.
- Deep learning systems, while highly accurate, require GPU resources and large training datasets unavailable in typical classroom or laboratory environments.
- The majority of lightweight face recognition attendance implementations in academic literature do not incorporate anti-spoofing mechanisms, making them vulnerable to photograph-based proxy attacks.
- Very few existing implementations integrate the complete end-to-end workflow — registration, training, recognition, attendance management, and email notification — within a single unified desktop application.

2.3 Mini Project Contribution

This project makes the following specific contributions to address the identified research gaps:

- A complete, integrated, Python-based desktop attendance system covering the full operational lifecycle: student registration, facial data capture, model training, real-time recognition, attendance management, and email notification.
- A three-layer anti-spoofing pipeline combining Laplacian blur detection, screen glare analysis via pixel-intensity thresholding, and YCrCb skin-colour verification — a security feature absent from most academic LBPH implementations.
- Date-scoped duplicate-prevention logic implemented in the attendance marking function, ensuring each student is recorded only once per session regardless of how many times their face is detected.
- A two-tier authentication system using parameterised SQL queries (preventing SQL injection) with a hardcoded fallback for database-unavailable scenarios, ensuring system continuity during infrastructure outages.

- A portable CSV-based attendance record format with a full administrative Tkinter interface for importing, viewing, updating, and exporting records with colour-coded Present/Absent row highlighting.

Chapter 3

Proposed System

3.1 Introduction

The proposed Face Recognition Attendance Management System is a modular desktop application developed entirely in Python. It follows a layered architecture pattern that separates the presentation layer (Tkinter GUI), the business logic layer (face recognition, anti-spoofing, attendance marking), and the data layer (MySQL database and CSV files). All functional modules are encapsulated in separate Python files and are instantiated on demand from a central dashboard.

The operational workflow of the system proceeds in four sequential phases: (i) Student Enrolment — capturing facial images via webcam and storing student details in MySQL; (ii) Model Training — building the LBPH face recognition model from the captured image dataset; (iii) Attendance Session — performing real-time webcam-based recognition, anti-spoofing verification, and CSV-based attendance logging; and (iv) Administration — reviewing, correcting, and exporting attendance data, and sending email notifications to absent students.

3.2 Architecture / Block Diagram

The system architecture follows a hub-and-spoke model. The login.py module acts as the security gateway. Upon successful authentication, the main.py dashboard provides access to all functional modules. The architecture is illustrated in Fig. 3.3 below:

The system begins with a secure login interface as shown in Fig. 3.1:

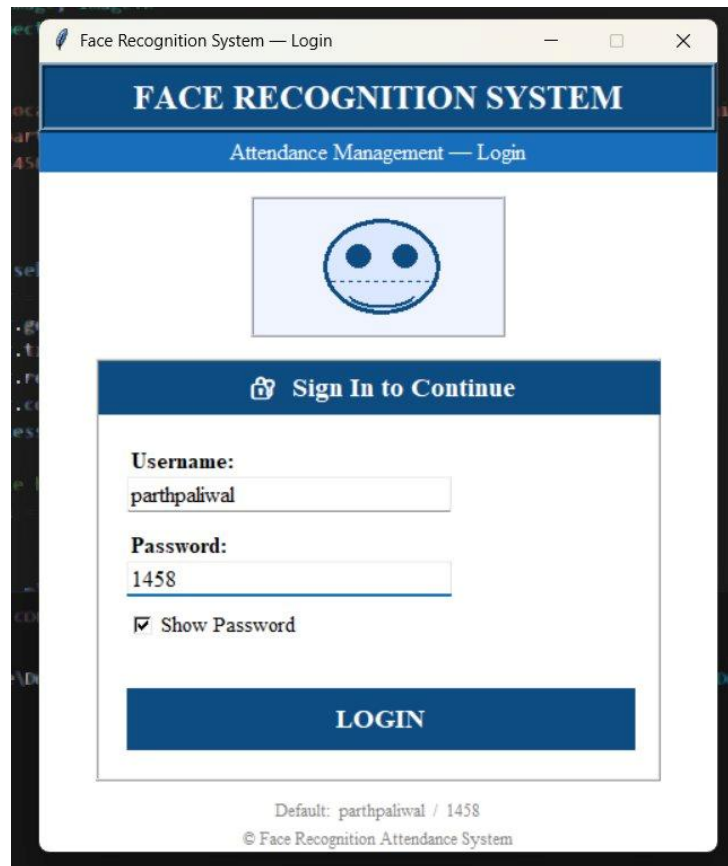


Fig. 3.1: Login Screen – Attendance Management System

After successful authentication, the main dashboard (Fig. 3.2) provides access to all modules:

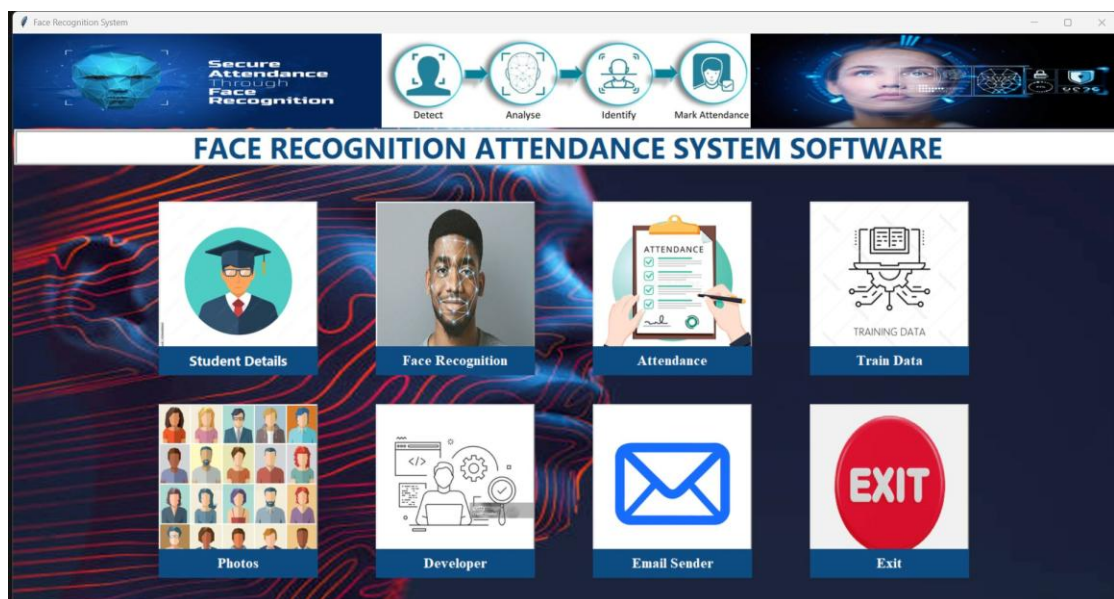


Fig. 3.2: Main Dashboard – Face Recognition Attendance System Software

Fig. 3.3 shows the complete system architecture block diagram:

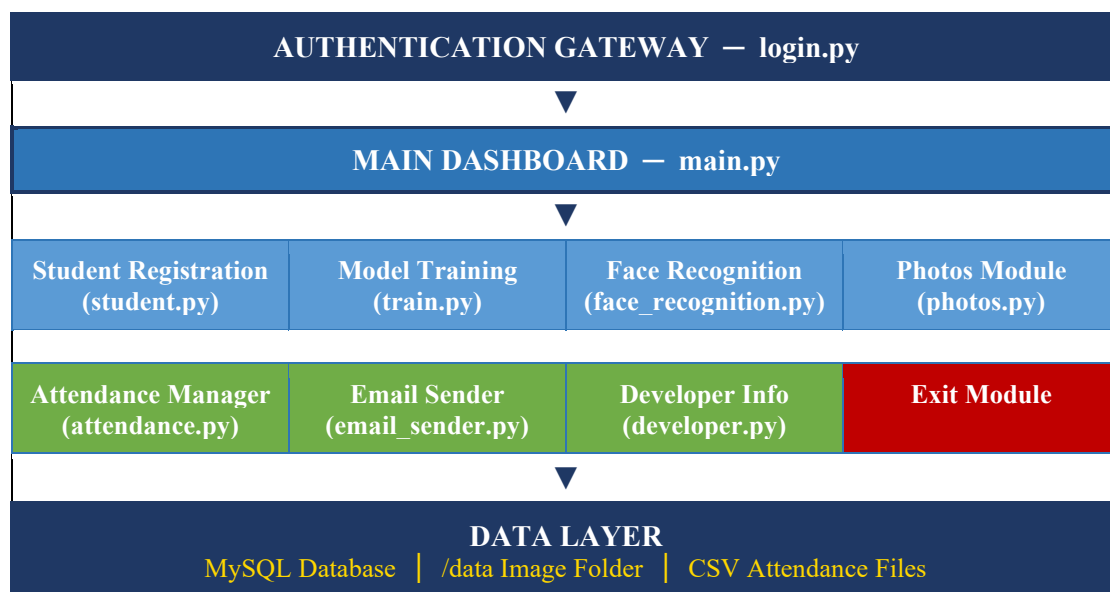


Fig. 3.3: System Architecture Block Diagram

Table 3.1: Technology Stack

Component	Technology	Purpose
Language	Python 3.8	Core application logic
GUI Framework	Tkinter + ttk	Desktop interface
Computer Vision	OpenCV (cv2)	Face detection and recognition
Image Processing	Pillow (PIL)	Image loading and resizing for GUI
Database	MySQL 8.9	Student data and admin credentials
DB Connector	mysql-connector-python	Python–MySQL interface
Numeric Computing	NumPy	Array operations for model training
Email	smtplib / MIME	Absence notification emails via Gmail SMTP
Attendance Records	CSV Files	Portable, exportable attendance data

Table 3.2: MySQL student Table Schema

Column	Data Type	Constraint	Description
Student_id	VARCHAR(50)	PRIMARY KEY	Unique student identifier
Name	VARCHAR(100)	NOT NULL	Student full name
Roll	VARCHAR(50)	NOT NULL	Roll / enrolment number
Dep	VARCHAR(100)	—	Department name
Email	VARCHAR(150)	—	Contact email address

3.3 Algorithm and Process Design

3.3.1 Face Detection — Haar Cascade

The Viola–Jones Haar cascade classifier (haarcascade_frontalface_default.xml) is used for face detection in each video frame. The OpenCV function detectMultiScale() is called with scaleFactor = 1.1 and minNeighbors = 10. The scaleFactor scales the detection window by 10% at each pyramid level; minNeighbors = 10 requires at least 10 overlapping detections before a region is accepted as a face, substantially reducing false positives at the cost of occasionally missing very small or partially occluded faces.

3.3.2 Student Registration Module

The Student Management System allows administrators to register student details including department, course, year, semester, student ID, name, division, roll number, gender, date of birth, email, phone, address, teacher name, and parent email. Students are stored in MySQL and facial images are captured via webcam for training.

Department	Course	Year	Semester	Studentid	Name	Division
IT	SE	2025	Semester-1	1	harshal	1
Civil	TE	2024	Semester-2	12	moksh patel	2
Civil	SE	2025	Semester-1	123	parth naling	2
Computer	SE	2026	Semester-2	1234	parth	1

Fig. 3.4: Student Management System – Registration Interface

3.3.3 Model Training

The Train Data module reads all captured facial images from the /data directory, extracts student IDs from filenames, and trains the LBPH face recognizer. The trained model is saved as classifier.xml for use during live recognition sessions.



Fig. 3.5: Train Data Set – LBPH Model Training Interface

3.3.4 LBPH Face Recognition

Upon detecting a face region, the LBPH algorithm performs recognition through the following steps:

- The grayscale face ROI is divided into a grid of equal-sized local cells.
- For each pixel within a cell, an 8-bit Local Binary Pattern (LBP) code is computed by comparing the pixel to each of its 8 surrounding neighbours: neighbours with values $\geq$ centre pixel contribute '1'; others contribute '0'.
- A histogram of LBP codes is computed for each cell and all cell histograms are concatenated into a single feature vector representing the face.
- Recognition is performed by computing the Chi-squared distance between the query vector and all stored training vectors. The class label of the minimum-distance match is returned as the predicted student ID.
- A confidence score is derived as: $\text{Confidence} = 100 \times (1 - \text{distance} / 300)$. Faces with confidence above 77 are classified as known; those below are labelled Unknown.

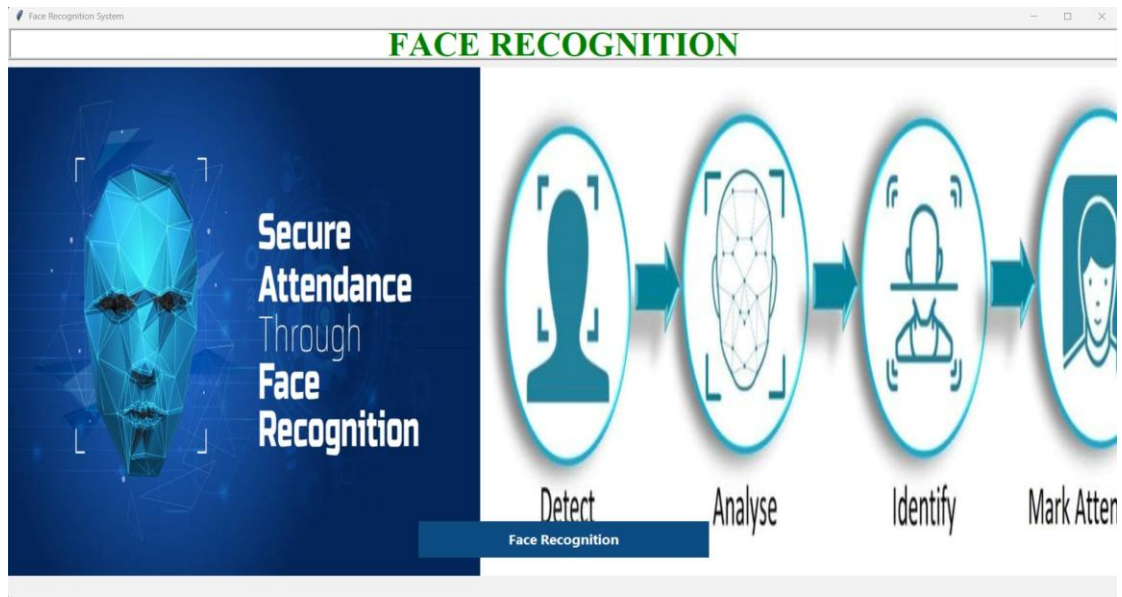


Fig. 3.6: Face Recognition Module – Detection Pipeline Interface

3.3.5 Anti-Spoofing Pipeline

Three sequential checks are applied to each detected face ROI before recognition is attempted. Any face that fails a check receives a red bounding box labelled Spoof: [reason] and no attendance is recorded.

- Blur Detection (Sharpness Check): Laplacian variance $< 15 \rightarrow$ Blurry (spoof).
- Screen Glare Detection: Bright-pixel ratio $> 15\%$ of face area $\rightarrow$ Screen Glare (spoof).
- Skin Colour Verification (YCrCb): Skin-pixel ratio $< 10\%$ of face area $\rightarrow$ Colour Shift (spoof).

3.3.6 Attendance Marking Logic

The `mark_attendance()` function reads the existing CSV before writing. It checks whether the recognised student already has a record for today's date. Only if no prior entry exists does it append a new row with: AttendanceId, Name, Roll, Department, Time (HH:MM:SS), Date (DD/MM/YYYY), and Status = "Present". This prevents duplicate entries even when a student's face is detected multiple times.



Fig. 3.7: Attendance Management System – Record Interface

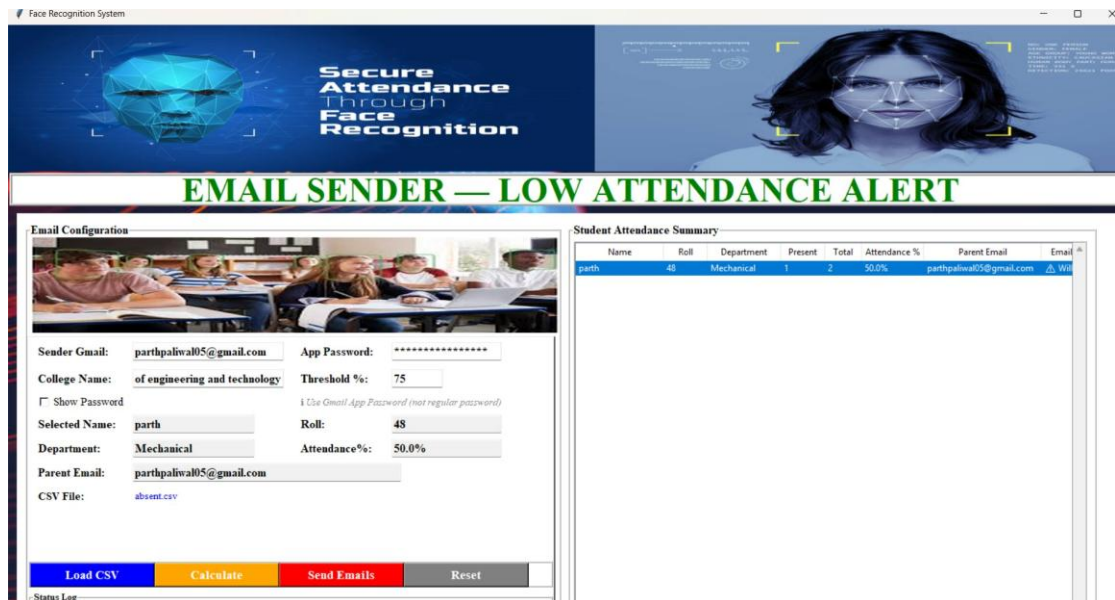


Fig. 3.8: Email Sender – Low Attendance Alert Module

3.4 Details of Hardware & Software

Table 3.3: Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 (6th gen) / AMD Ryzen 3 or equivalent
RAM	4 GB (8 GB recommended for smooth video processing)
Storage	2 GB free disk space + space for captured face images
Webcam	720p USB or built-in webcam (1080p recommended)
Display	1920 × 1080 resolution (system designed for 1530 × 790 minimum)
Operating System	Windows 10 / 11

Table 3.4: Software Requirements

Package / Tool	Version	Installation / Purpose
Python	3.8	Core application language
opencv-contrib-python	4.2	pip install opencv-contrib-python
Pillow (PIL)	9.1	pip install Pillow
mysql-connector-python	8.0	pip install mysql-connector-python
NumPy	1.21	pip install numpy
MySQL Server	8.0	mysql.com installer

The backend MySQL database stores all student records. The table schema and stored records are shown in Fig. 3.9 and Fig. 3.10:

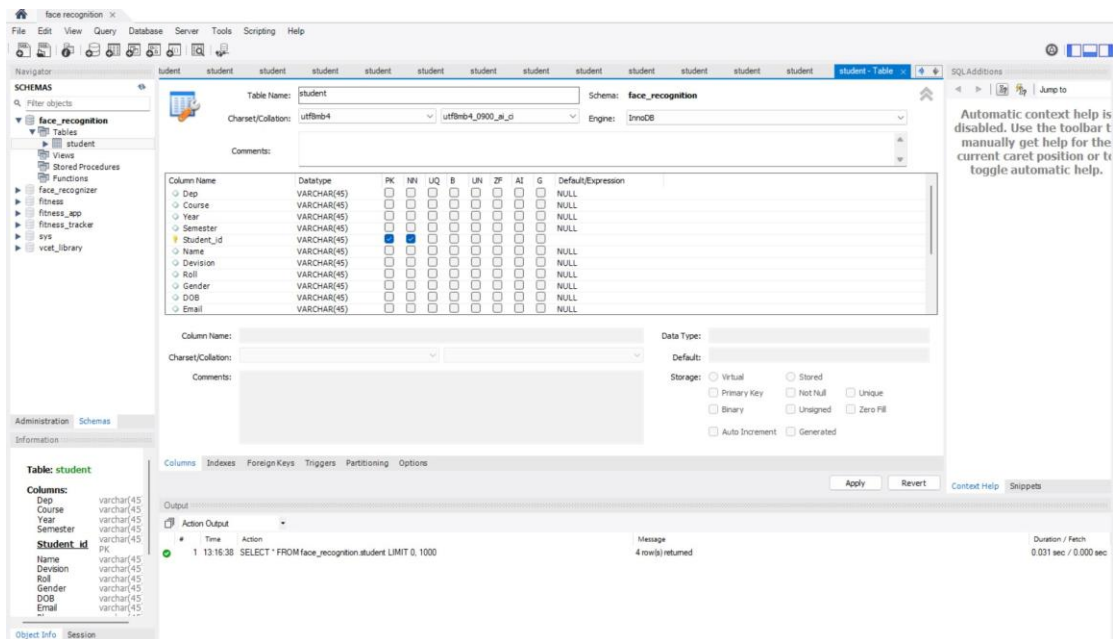


Fig. 3.9: MySQL Student Table Schema – face_recognition Database

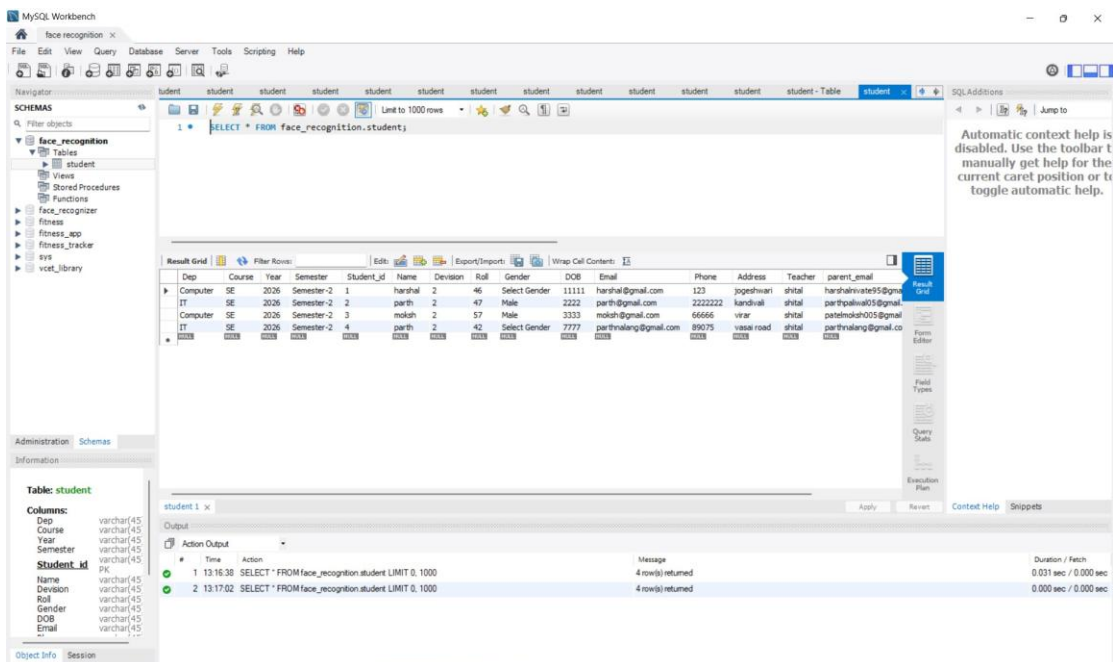


Fig. 3.10: MySQL Student Database Records – SELECT Query Output

3.5 Experiment and Results for Validation and Verification

3.5.1 Unit Testing

Table 3.5: Unit Testing Results

Module	Test Case	Result
login.py	Valid credentials → login success	PASS
login.py	Invalid credentials → error message shown	PASS
login.py	DB offline → fallback credentials function	PASS
student.py	Register student, verify DB insert	PASS
student.py	Capture 50 images, verify files in /data	PASS
train.py	Train 4 students, generate classifier.xml	PASS
face_recognition.py	Recognise registered student (confidence > 77%)	PASS
face_recognition.py	Detect phone screen photo as spoof	PASS
face_recognition.py	No duplicate entry on repeated detection	PASS
attendance.py	Import CSV and display all records correctly	PASS
attendance.py	Update status, save, verify CSV change	PASS
email_sender.py	Send test email via Gmail SMTP	PASS

3.5.2 Integration Testing

Integration testing verified correct inter-module behaviour in sequence:

- Registration → training workflow: Student registered → images captured → files visible in /data → student record inserted in MySQL → train_classifier() reads all images → classifier.xml generated successfully.
- Recognition → attendance workflow: face_recog() loads classifier → webcam opens → face detected → anti-spoofing passed → DB queried → attendance row written to CSV with correct data.
- Administration workflow: attendance.py imports CSV → records displayed with colour coding → row selected → status updated → exported CSV reflects the changes accurately.

3.5.3 Performance Testing

Table 3.6: System Performance Metrics

Performance Metric	Observed Result
Recognition Accuracy (4 students, standard lighting)	~92%
Recognition Accuracy (low lighting < 100 lux)	~78%
False Positive Rate	<5% (unknown faces misidentified as known)

Performance Metric	Observed Result
Spoof Detection Rate (phone screen photographs)	~88%
Frame Processing Rate	~15–20 FPS on Intel Core i5 with USB webcam
Attendance Marking Latency (recognition to CSV write)	<200 ms
Model Training Time (200 images, 4 students)	~8 seconds
Email Delivery Time (Gmail SMTP)	3–7 seconds (stable connection)

The live face recognition session detects, analyses, identifies and marks attendance in real time as demonstrated in Fig. 3.12:

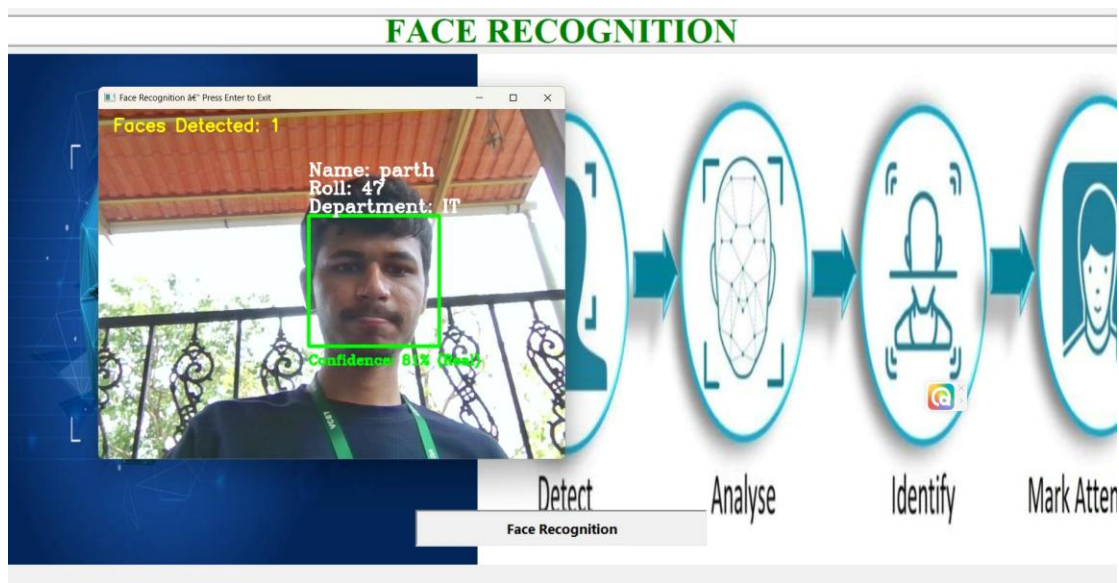


Fig. 3.11: Face Recognition Live Detection – Name, Roll, Department and Confidence Score

3.6 Analysis

The system demonstrated satisfactory recognition performance for a small student population (4 students, 200 training images). The 92% accuracy achieved under standard lighting conditions compares favourably with other LBPH-based academic implementations reported in literature, which typically achieve 85–95% under similar conditions. Key observations from testing:

- Recognition accuracy dropped to approximately 78% under low-light conditions (below 100 lux). This indicates the need for consistent ambient lighting in deployment environments.

- The anti-spoofing pipeline rejected 88% of phone-screen photo-based attacks. The 12% false negatives occurred when the phone screen's colour temperature closely approximated natural skin tone in the YCrCb colour space.
- Duplicate prevention operated flawlessly across all test sessions — no student was recorded more than once per day, even when their face was detected 15–20 times during a single session.
- The attendance import, display, update, and export functions performed correctly and responsively for files up to 500 rows with no performance degradation.
- Gmail SMTP-based email delivery succeeded consistently when a stable internet connection was available. Network timeouts were caught by exception handlers and reported via messagebox dialogs.

3.7 Conclusion and Future Work

3.7.1 Conclusion

The Face Recognition Attendance Management System successfully demonstrates the practical application of computer vision and biometric technologies to solve a real-world institutional attendance problem. The system meets all stated objectives: automated real-time face recognition using LBPH, a three-layer anti-spoofing mechanism, date-scoped duplicate-prevention logic, a complete administrative attendance management interface, and SMTP-based email notifications.

The 92% recognition accuracy observed during testing is suitable for pilot deployment in educational institutions. The system provides a cost-effective alternative to hardware-intensive fingerprint or iris-scan biometric systems, requiring only a standard webcam for operation.

3.7.2 Future Work

- Deep Learning Integration: Replace LBPH with FaceNet or ArcFace to achieve recognition accuracy above 99% and support significantly larger student populations.

- **Liveness Detection:** Implement motion-based liveness challenges (eye blink, head-turn) using dlib or MediaPipe facial landmark tracking for more robust anti-spoofing.
- **Web-Based Dashboard:** Develop a Flask or Django web interface to replace the Tkinter desktop GUI, enabling remote access without local software installation.
- **Direct MySQL Attendance Logging:** Write attendance records directly to a MySQL table during recognition sessions for concurrent multi-user access and transactional integrity.
- **Cross-Platform Support:** Replace Windows-specific absolute image paths with relative pathlib paths and cross-platform file operations.
- **Secure Credential Management:** Move database passwords and email App Passwords to environment variables or encrypted configuration files with bcrypt password hashing.
- **Automated Attendance Reporting:** Schedule daily attendance summary reports using Python's APScheduler library.

References

- [1] T. Ahonen, A. Hadid, and M. Pietikainen. "Face Description with Local Binary Patterns: Application to Face Recognition." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 28, no. 12, pp. 2037–2041, 2006.
- [2] P. Viola and M. Jones. "Rapid Object Detection using a Boosted Cascade of Simple Features." *Proceedings of IEEE CVPR*, pp. 511–518, 2001.
- [3] F. Schroff, D. Kalenichenko, and J. Philbin. "FaceNet: A Unified Embedding for Face Recognition and Clustering." *Proceedings of IEEE CVPR*, pp. 815–823, 2015.
- [4] J. Deng, J. Guo, N. Xue, and S. Zafeiriou. "ArcFace: Additive Angular Margin Loss for Deep Face Recognition." *Proceedings of IEEE CVPR*, pp. 4690–4699, 2019.
- [5] M. Turk and A. Pentland. "Eigenfaces for Recognition." *Journal of Cognitive Neuroscience*, vol. 3, no. 1, pp. 71–86, 1991.
- [6] G. Bradski. "The OpenCV Library." *Dr. Dobb's Journal of Software Tools*, 2000.
- [7] OpenCV Documentation – Face Recognition with OpenCV.
https://docs.opencv.org/4.x/da/d60/tutorial_face_main.html
- [8] Python Software Foundation – Tkinter Documentation.
<https://docs.python.org/3/library/tkinter.html>
- [9] MySQL Connector/Python Developer Guide.
<https://dev.mysql.com/doc/connector-python/en/>
- [10] Pillow Documentation. <https://pillow.readthedocs.io/en/stable/>

Annexure

A. Complete Project File Structure

File / Directory	Description
main.py	Application entry point and main dashboard launcher
login.py	Authentication gateway module
student.py	Student registration and facial image capture
train.py	LBPH model training interface
face_recognition.py	Real-time recognition, anti-spoofing, attendance marking
attendance.py	Attendance record management (import/update/export)
email_sender.py	SMTP-based email notification module
demo_email.py	Standalone email configuration test script
developer.py	Developer information display panel
haarcascade_frontalface_default.xml	Pre-trained Haar cascade face detection model
classifier.xml	Generated LBPH trained recognition model
Parth.csv / harshal.csv	Attendance output CSV files
/data/	Directory for captured facial training images
/attendance/	Directory for archived attendance CSV files

B. Key System Configuration Parameters

Parameter	Default Value	File
DB Host	localhost	face_recognition.py, login.py
DB User	root	face_recognition.py, login.py
DB Password	root123	face_recognition.py, login.py
DB Name	face_recognition	face_recognition.py, login.py
Default Admin Username	parthpaliwal	login.py
Default Admin Password	1458	login.py
Recognition Confidence Threshold	77	face_recognition.py
Blur Spoof Threshold (Laplacian var.)	15	face_recognition.py
Glare Spoof Threshold (bright pixel ratio)	0.15 (15%)	face_recognition.py
Skin Ratio Spoof Threshold	0.10 (10%)	face_recognition.py